AI ACCELERATOR · COHORT 5 · WEEK 6 CAPSTONE

From Raw Ledgers to an FY2027 Budget Engine: Mastery Submission Guide

Kurt Cherrington, CFO · October 2026 · Companion to Consolidation_Pipeline_Demo.ipynb, Consolidation_Output_Demo.xlsx, FY2027_Baseline_Forecast_Demo.xlsx and the FY2027 scenario app

1. Submission at a glance

Item	Detail
Level	Mastery (state it in the first sentence of the Loom)
Problem	Our firm's first real budget (FY2027) has to be built from about 31 legal entities kept in two ledgers (Yardi and QuickBooks Online), with intercompany fees, partial months, a month-end close being rebuilt and properties bought mid-year. Consolidating and tying it by hand takes days each month, and nothing tells Asset Management what a normal year looks like.
Solution	Three linked pieces of Python: **BudgetPipeline** loads raw Yardi and QBO exports, cleans them to a written rulebook, detects accounting anomalies, consolidates with intercompany elimination and checks itself. **BaselineForecaster** turns the cleaned actuals into an FY2027 momentum baseline with the Week 5 method, extended to every property type. The **Streamlit app** lets the CFO move six drivers and watch the budget update.
Data	The submission runs on a fictional company, Juniper Ridge Partners: 10 files in the real Yardi and QBO layouts, 13 entities, 25 planted problems. Every name and number is invented. The same code ran on our firm's 34 real exports on Oct 24, in approved tools only.
Pipeline results	Demo: 25 of 25 planted problems caught. Real run: 3,042 of 3,042 source totals tie, 297 of 297 balance-sheet checks, 86 of 86 earnings ties, $0 unmapped.
FY2027 baseline	8 properties (demo): revenue $11.36M, property NOI $5.36M, 80% range $4.95M to $5.76M (TTM as booked: $5.29M). Backtest error: 5.9% for seasonal assets (peer-borrowing) and 1.5% for steady assets (run-rate), vs 74% for Prophet on a property's own months.
Scenario app (Base)	Consolidated revenue $14.06M, NOI $5.36M, net income $1.00M, fees to the management company $2.20M. NOI ranges from $3.65M (Low) to $7.42M (High).
Deliverables	Colab notebook (48 cells, runs end to end), consolidation workbook (20 sheets), FY2027 baseline workbook (8 sheets), Streamlit app (GitHub + Streamlit Cloud), this guide, Loom of about 6 minutes
Links	Live app (Streamlit Cloud): https://fy2027-budget-scenarios-nvx3sn4bhmgkrtyz5vgpjt.streamlit.app/ Code and submission files (GitHub): https://github.com/kurtcherrington-boop/fy2027-budget-scenarios Backup copy of the submission files (Google Drive): https://drive.google.com/drive/folders/1UlcjnGKsTGNs0rXDICw5qK7F902WrZnh?usp=drive_link

Rubric map: where each Mastery criterion is evidenced (7 of 7; 6 needed to pass)

Criterion (Mastery bar)	Evidence	Where	Loom
1. Complex problem identified	First-ever budget built from 31 entities, two ledgers, intercompany flows, an unfinished close and mid-year acquisitions. Today this takes days of manual work a month.	Guide §2; notebook intro	0:00–0:45
2. Data cleaning logic defined	Cleaning_Logic_v0.1 (rules R-01 to R-14) was written, reviewed and approved before any code (Oct 23). It covers dates, nulls, text in number fields, structure, keys, sources and the accounting rules.	Guide §3; notebook §0	0:45–1:30
3. Working Python code	The notebook runs top to bottom in Colab with 0 errors and produces clean DataFrames: bp.data,	Notebook §1–6	1:30–2:15

Criterion (Mastery bar)	Evidence	Where	Loom
	bp.normalized() and fc.fc.		
4. Reusable data pipeline	Two classes, BudgetPipeline and BaselineForecaster, in a 12-module package with a config workbook. Next month: drop in the new exports and Run all. 73 automated tests.	Guide §4	1:30–2:15
5. Model trade-off explained	Seven approaches backtested per asset family, with a written selection rule. The accuracy vs interpretability discussion is written for a CFO, and the CFO memo is generated from the numbers.	Guide §5; notebook §4c	2:15–3:15
6. Interactive application	Streamlit app with six sliders, Low/Base/High presets, a driver bridge, the baseline forecast with its 80% band, and funds, properties and data-quality views.	Guide §6; app link	3:15–4:15
7. Scalability addressed	Measured at 2× and 10× data: the pipeline and app are linear, the backtest doubles, and the first thing to break is the people process, not compute.	Guide §7	4:15–4:40

2. The problem (Criterion 1)

FY2027 is our firm's first true budget. The firm runs three funds, a management company and RV parks, apartments, self storage and an industrial park. That is about 31 legal entities, kept in **two ledgers**: Yardi for most properties and the funds, and QuickBooks Online for the RV parks. The month-end close is being rebuilt, so recent months can be partial. Several properties were bought mid-year. Management and asset-management fees flow between the funds, the properties and the management company, and must tie and be eliminated.

Before this project, producing one consolidated, trustworthy trailing-12-month P&L meant downloading dozens of exports and mapping two charts of accounts by hand. Then came tying intercompany fees and finding the accruals, catch-ups, capital items and stopped books by eye. That was days of work every month, and it was repeated for every budget scenario.

Friction today	What the prototype does in minutes
Dozens of exports in four layouts (Yardi P&L and balance sheet, QBO P&L and balance sheet)	Reads every layout, ties every file to its own totals, and checks that every entity has uploaded every month
Two charts of accounts; the same code can mean different accounts	Maps each ledger with its own rules to one set of budget lines (Line IDs); one book of record per entity-month, so nothing is double counted
Anomalies found by eye, if at all	Twelve explainable detectors propose reason-coded adjustments; the CFO approves anything over $5,000 or 5%
No reference point for the budget	An FY2027 momentum baseline per property and month, with an 80% range, selected by backtest
Every what-if is a spreadsheet rebuild	Six sliders; every number and the bridge update immediately

Why it is a wicked problem. The data is wrong in different ways in each system. The rules must be explicit enough for an auditor to follow, and the output has to be trusted by a CEO and GPs who will never read the code.

3. Data cleaning logic, written before code (Criterion 2)

The rulebook (Cleaning_Logic_v0.1) was drafted from a read-only diagnosis of the real files (16 findings, H-01 to H-16) and approved on Oct 23 before the pipeline was written. It has two layers. **Layer A** makes the data readable. **Layer B** normalizes the accounting. Raw data is never edited: every change is a reason-coded row in an adjustments register, and nothing is filled silently.

Rule	How it handles the data
R-01 Structure	Reads detail rows by account-code pattern and never reads subtotals as data. Rebuilds every subtotal and NET INCOME line and compares it with the source total. One bad cell can't cascade (a total ties to the header that matches in most months).
R-02 Dates	Reads "Jul 2025", "Jul-25", "July 2025", "2025-07", "07/2025", "7/31/2025", Excel serials and "M07" + year. FY, Total and T-12 columns are never months. A partial month ("Oct 1-24 2026") is dropped and logged.
R-03 Text in numbers	Converts "$1,234.50", "(1,234)", "1,234-" and "-". "#REF!" and "n/a" are left blank, not zero, and logged as High. One sign convention: revenue +, expenses −.
R-04 Nulls	Each entity has a window from acquisition to last closed month. An empty month is classed as actual zero, closed season, awaiting feed (stopped books) or not owned. Only actual zeros count as zero; the forecaster treats the others as gaps.
R-05 Keys	Joins on SCODE + Line ID, never on entity name. Each ledger maps with its own rules; unmapped accounts stay in on an UNMAPPED line and keep the run provisional.

Rule	How it handles the data
R-06 Sources	One book of record per entity-month (QBO > Yardi > flat by default; Yardi for the one park whose QBO books are partial). Amortizing items Yardi carries before QBO was set up are supplemented, never double counted.
R-07 to R-10 Detectors	D01–D12: missing accruals and catch-ups, missing revenue, costs to amortize, capital items in repairs, stopped books, missing expense categories, deposits to defer, reversals and duplicates, intercompany breaks.
R-11 Register	Reason codes MISSING, TIMING, CLASS, OTHER; status Proposed / Approved / Rejected. Normalized = raw + approved. The CFO rules on anything over $5,000 or 5% of the line.
R-12 Intercompany	Each pair ties by quarter (fees are booked Day 1 of each quarter) and is eliminated; an untied amount stays visible and keeps the run provisional.
R-13 Scope	Facts at 100% of each asset, with an at-share view from ownership %. An excluded asset stays out of every measure.
R-14 Checks	Every check shows OK or the difference to three decimals; any open check stamps every output PROVISIONAL.

What I would do in real life (and did). Missing uploads stop the run until they arrive, and the notebook lists them by entity and month. Where Yardi and QBO disagree, I ruled on the book of record per entity. Anything a detector proposes over the threshold waits for my approval. The forecast in production uses approved adjustments only.

4. Working Python and a reusable pipeline (Criteria 3 and 4)

Everything is a package (budget_pipeline/, 12 modules) that the notebook writes to disk in step 0, so the notebook is self-contained in Colab. The flow:

Upload → completeness gate → clean and map → detect → consolidate and eliminate → check → BIWS workbook → FY2027 baseline forecast → app data → Streamlit app

Class / module	What it does	Reuse next month
BudgetPipeline (pipeline.py)	add_file() detects each layout per sheet; run() cleans, maps, detects, consolidates and checks; export() writes the BIWS workbook; export_app_data() writes the app's files	Drop the new exports in the upload, set LAST_CLOSED, Run all
BaselineForecaster (forecast.py)	prepare() classifies every account; backtest() scores 7 approaches per asset family; build() makes FY2027 Low/Base/High by property, line and month; export() writes the baseline workbook	Re-selects its approach on the new history automatically; a second year of data makes local seasonal models eligible
pipeline_config.xlsx	Entity master, chart mapping, intercompany map, thresholds: yellow input cells, no code changes	A new entity or GL account is one row
Tests	52 pipeline and forecaster tests (each layout, each detector, book of record, the forecast rules), a 25-problem planted answer key, and 21 model and app tests (the bridge adds up, intercompany cancels, the forecast is the starting point)	Run on every change

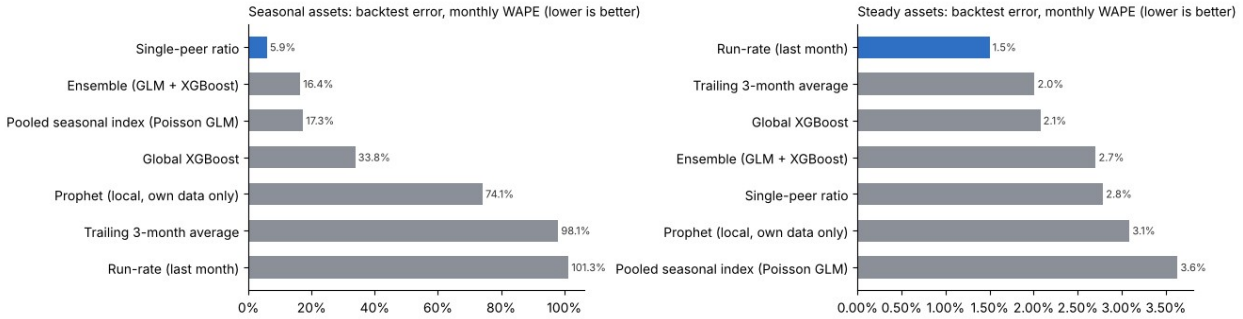
Proof it works. The demo notebook runs end to end with 0 errors. Step 6 scores the run against the planted answer key and catches 25 of 25 planted problems: every detector D01–D12, dirty cells, a partial month, a roll-up tab, two ledgers for one property, and missing, unbalanced and untied balance sheets. On our real exports (Oct 24), every file tied to its own totals: 3,042 of 3,042. The workbook recalculates with 0 formula errors.

5. The forecast and the model trade-off (Criterion 5)

Question. If every property repeats its trailing-12-month operations, what revenue and NOI does each produce in FY2027, by month? How sure are we, and where must Asset Management supply the answer instead of a model?

Most months are "same month last year". The hard part is the months a property has **not** operated under our ownership. Bluewater (bought in May) has 5 of 12 months, and Elm Street's September books stopped. Week 5 showed that no model can learn a season from a property's own few months, and that borrowing the seasonal shape of peers works. Week 6 turns that into a class that runs on the pipeline's output for every asset type.

Step	Rule
Input	The normalized baseline (raw + approved adjustments), so the forecast inherits the cleaning: the $48K roof moves to capex, the annual insurance premium is spread, the campground's deposit is deferred, and the missing property tax is accrued
Lines	Contract revenue (scheduled), transient & ancillary revenue (demand), COGS (% of demand), payroll and utilities (seasonal profile), other opex (run-rate), management fees (% of revenue), below NOI (repeated, not forecast)
Families	Seasonal: RV parks, campground, marina. Steady: apartments, storage, industrial. Peers are borrowed within a family only
Backtest	Keep a 3- or 5-month window of a complete property (as if just bought), predict its other months, score against actuals: 429 test months per approach (210 seasonal, 219 steady)
Selection	Bias within ±10%, lowest monthly WAPE, simpler approach within 0.5 pp wins
Ranges	80%: own months ±10%; modelled months ± the selected approach's backtest error

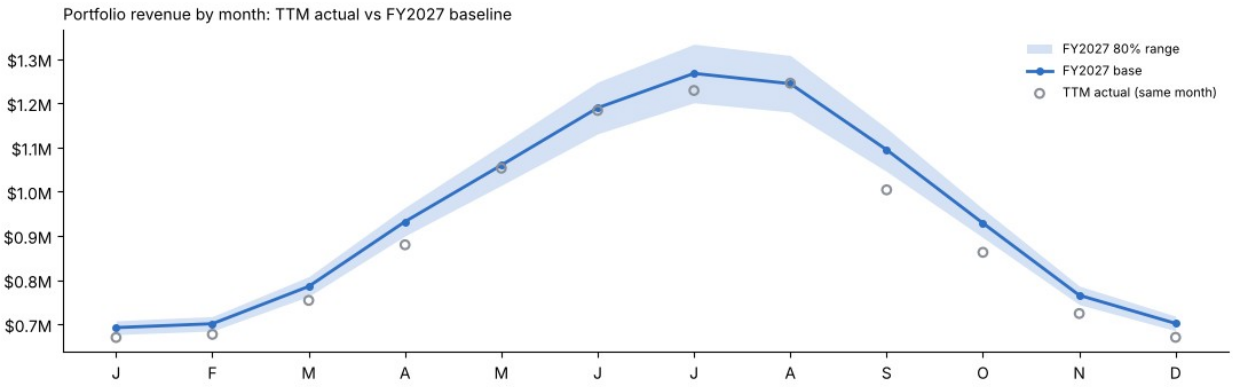


Backtest error by approach and family. Blue = selected by the written rule.

Approach	Learns from	Seasonal WAPE	Seasonal bias	Steady WAPE	Complexity
Single-peer ratio	one comparable property	**5.9% (selected)**	−2.1%	2.8%	2
Ensemble (GLM + XGBoost)	all peers	16.4%	−8.6%	2.7%	5
Pooled seasonal index (Poisson GLM)	all peers	17.3%	+7.6%	3.6%	3
Global XGBoost	all peers	33.8%	−24.8%	2.1%	5
Prophet (local, own data only)	the property itself	74.1%	−32.0%	3.1%	4
Trailing 3-month average	the property itself	98.1%	−8.4%	2.0%	1
Run-rate (last month)	the property itself	101.3%	+7.5%	**1.5% (selected)**	1

Accuracy vs interpretability, for a CFO

- **The accurate answer and the explainable answer are the same here.** For seasonal assets, scaling one comparable property's curve to our own months (5.9%) beat the pooled GLM (17.3%), XGBoost (33.8%) and Prophet (74.1%). Each number has a one-sentence explanation: "same month last year, or its peer's shape scaled to its own months."
- **Why not the machine-learning models?** With 8 properties and 12 months, XGBoost has too little data to learn from and can't explain a single month's number to a board. It would need to win clearly to earn its place, and it didn't. The written rule makes that a policy, not a preference: a simpler approach within 0.5 pp wins.
- **Why not Prophet?** With fewer than 24 months it can't fit a yearly season, so it extrapolates a trend: −32% bias. It's the right tool with long history and the wrong one here.
- **Steady assets need no season.** A rent roll repeats at its latest rate, so the run-rate wins (1.5%). It also captures Elm Street's lease-up, which "same month last year" would miss.
- **What I gave up.** The ±10% band on own months is a judgment: one year can't measure year-over-year variation. The scenario drivers are pure arithmetic, so they can't learn elasticities such as occupancy against rate. Both are deliberate: the CEO and GPs can defend every bar of the bridge.
- **It changed the answer.** The app's earlier shortcut grossed partial-year properties up by a peer ratio. The line-by-line forecast puts Base net income **$0.25M lower** ($1.00M vs $1.25M), because winter months carry costs and interest but little revenue. A summer-only P&L overstated Bluewater's NOI three-fold ($190K for 5 months vs $59K for a full year).
- **Honest caveat.** The demo parks share one seasonal shape by construction, so peer-borrowing scores better here than on real books. On real data in Week 5 it scored 36%, still far ahead of the property-only methods at 90% or more.



Portfolio revenue by month: TTM actual (open circles) vs FY2027 base and 80% range.

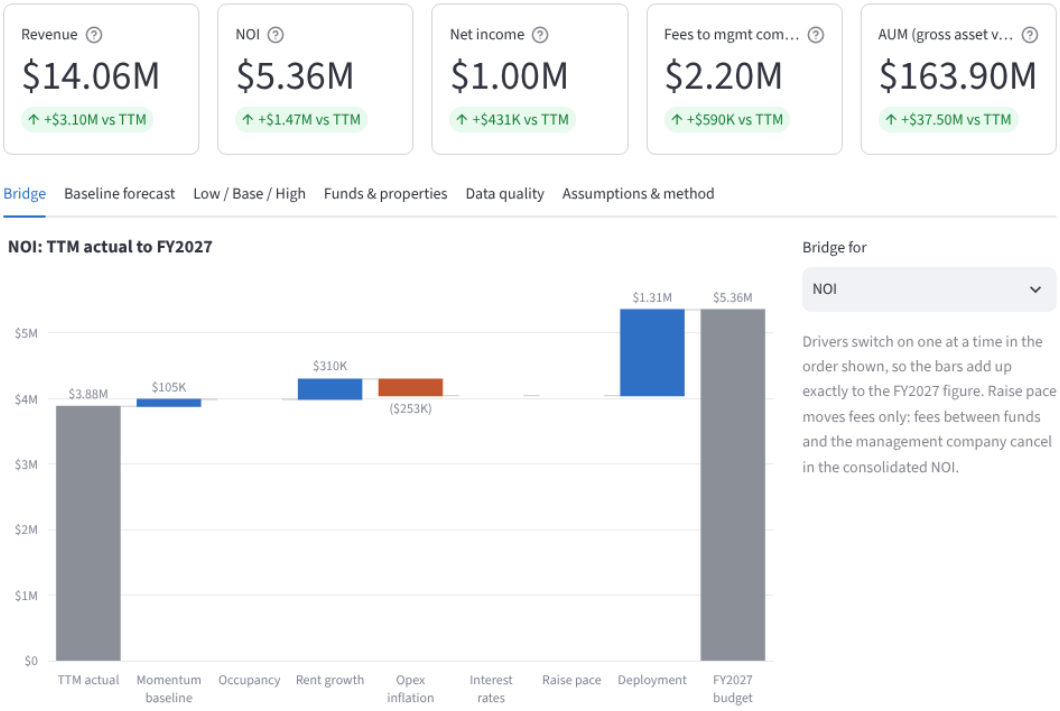
Property	Family	Months seen	Revenue base	NOI low	NOI base	NOI high	Borrowed
Aspen Hollow RV Resort	Seasonal	12	$1,298K	$154K	$230K	$306K	0%
Maple Court Apartments	Steady	12	$2,098K	$1,011K	$1,051K	$1,091K	0%
Stor-Easy Storage	Steady	12	$1,075K	$639K	$659K	$679K	0%
Riverside Business Park	Steady	12	$2,878K	$2,247K	$2,303K	$2,358K	0%
Elm Street Lofts	Steady	11	$1,088K	$478K	$499K	$520K	8%
Pinecrest RV Park	Seasonal	12	$1,637K	$404K	$509K	$614K	0%
Bluewater Marina & RV	Seasonal	5	$654K	$8K	$59K	$111K	25%
Cedar Bluff Campground	Seasonal	12	$636K	$4K	$45K	$86K	0%

Property	Family	Months seen	Revenue base	NOI low	NOI base	NOI high	Borrowed
Total			**$11,364K**	**$4,947K**	**$5,355K**	**$5,763K**	**2%**

Property NOI is after management fees and before debt service. Property-level figures are unaudited.

6. The interactive application (Criterion 6)

The Streamlit app starts from the FY2027 momentum baseline and applies six drivers in a fixed order: occupancy, rent growth, opex inflation, interest rates, raise pace and deployment. It is built as two files: budget_model.py holds the math (plain Python, unit tested) and app.py draws the screen. Every slider move re-runs the script and redraws every number.



KPI tiles and the NOI bridge: TTM actual → momentum baseline → each driver → FY2027. The bars add up exactly (tested).

Tab	What the CFO sees
Bridge	A waterfall for NOI, net income or fees to the management company. Blue bars help, rust bars hurt.
Baseline forecast	Monthly revenue or NOI for the portfolio or any property, with the FY2027 base, 80% band and TTM actual; the by-property table; the backtest leaderboard.
Low / Base / High	The three presets beside your scenario. Base = 50% chance each assumption is too high and 50% too low.
Funds & properties	NOI by fund and property, at 100% or at ownership share; CSV download of the scenario by entity and line.
Data quality	The pipeline's checks and findings. A PROVISIONAL banner shows while any check is open.
Assumptions & method	Editable property assumptions (occupancy, asset value, debt, floating share) and the method, step by step.

Scenario	Occ.	Rent	Opex	Rates	Raise	Deploy	Revenue	NOI	Net income
Low	−3 pts	1%	5.0%	+100 bp	$10M	40%	$11.66M	$3.65M	($0.40M)
Base	0	3%	3.5%	0	$25M	60%	**$14.06M**	**$5.36M**	**$1.00M**
High	+2 pts	5%	2.5%	−50 bp	$40M	80%	$17.27M	$7.42M	$2.46M

Consolidated: intercompany fees eliminated; fund and management-company costs included. It's a coincidence that Base consolidated NOI ($5.36M) is close to the property NOI baseline ($5.36M): they measure different things.



The Baseline forecast tab: FY2027 base, 80% range and TTM actual by month.

7. Scalability: what breaks if the data doubles? (Criterion 7)

For us, data doubling isn't hypothetical: it's the growth plan. The firm targets $1B of AUM by 2030, and every new property, fund or SPV is another set of ledgers to consolidate. So the question is what breaks along that path, and when.

Measured, not assumed. The demo was run at its normal size, with every property cloned (2×), and with the app's baseline replicated 10×.

Component	Today (demo)	At 2×	What breaks first, and the fix
Load + clean + detect + consolidate	3.5 s, 6,004 rows	Linear (~7 s)	Nothing at 2× or 10×. pandas holds millions of rows; Colab has 12 GB of RAM. Many files: upload one zip (built in).
BIWS workbook export	4.0 s, 0 formula errors	~8 s to write	Opening it: the schedules are live SUMIFS over the Data tab, so recalculation work grows with rows × formula cells (about 4× at 2× entities and months). Fix at ~10×: split by fund, or move the schedules to Power Query or pivots. Excel's 1,048,576-row limit is about 100× away.
Forecast backtest	40 s, 429 test months × 7 approaches	80 s, 858 test months × 7 (measured)	Linear in properties; Prophet is most of the time. Fix: drop Prophet from routine runs once it keeps losing, run the 7 approaches in parallel, and backtest at month-end close only.
Scenario app	0.15 s per slider move	0.15 s (10×: 0.23 s)	Nothing for this firm. Streamlit Community Cloud caps memory at 1 GB: fine for the demo. Real data runs locally.
People and process	1 reviewer	2× findings to rule on	This breaks first. Every new entity or GL account needs a config row: the completeness gate stops the run until it's added (it caught a new management-company entity on the real run). CFO-

Component	Today (demo)	At 2×	What breaks first, and the fix
			ruled findings double, so keep the $5K / 5% threshold and bulk-approve the small ones.
Model quality	8 properties	16 properties	Improves: more peers make borrowing more accurate, and the pooled GLM is built to overtake the single-peer ratio as the portfolio grows. The written rule switches automatically when it wins.

8. Security: how PII was handled

- **There is no personal data in scope.** The pipeline reads property and fund P&Ls and balance sheets. Payroll appears only as account totals; tenant ledgers, investor records and bank details are never loaded.
- **The submission data is synthetic, not masked.** Juniper Ridge Partners is generated from seeded random numbers in the real file layouts, so nothing can be re-identified. Every file, notebook output, workbook and README was scanned against a block list of real names, with 0 hits. The block list and anonymizer stay internal and are not submitted.
- **Prompts carried structure, not records.** SNAKE prompts described layouts, rules and constraints, never investor, tenant or employee names. Real exports were processed only in approved tools: Claude inside the company's Claude Project, my OneDrive, and Colab under my documented approval as CFO (Oct 22).
- **The app splits public and private.** The public Streamlit link and its GitHub repo hold only the demo. Real data runs on localhost: the file uploader reads a local export, and nothing is sent anywhere. There are no credentials in the code.
- **A human stays in the loop.** Detectors only propose; the CFO approves anything over $5,000 or 5%. Outputs are stamped PROVISIONAL while any check is open, and property figures are unaudited (the funds are audited annually).

Governance gate	Status
1. Initiation	Owner: Kurt Cherrington (CFO). AI Use Case Register entry: consolidation pipeline + baseline forecaster + scenario app
2. Risk assessment	Medium: sensitive internal financials with human review; no PII; no automated postings to the ledger
3. Pre-deployment validation	Real tie-outs (3,042 of 3,042 totals, 297 of 297 balance sheets), 25 of 25 planted problems, 73 tests, 0 formula errors. Fallback: the existing manual close
4. Ongoing review	Quarterly. The prompt log records each prompt, the model (claude-opus-5-5) and its validation

9. ROI: hours of human labor automated per month

Position: 18 hours to build; about 20 hours a month saved, net of upkeep; it pays for itself on the first real consolidation run. These are estimates, built bottom-up from the work as it is done today, and they will be measured from the first real run on Nov 3.

Base case

Task	Manual today	With the prototype	Hours saved
Consolidation run in budget season (4 runs between the Nov 3 checkpoint and the Dec 15 lock)	30 h per run	4 h per run (exports, rulings on flagged items and sign-off stay manual)	104 h per cycle
Monthly refresh after lock: rolling forecast plus the	16 h a month	4 h a month (same pipeline;	144 h a year

Task	Manual today	With the prototype	Hours saved
budget-vs-actual pack		commentary stays manual)	
Upkeep: config rows for new entities and accounts, code fixes	–	1 h a month	(12 h a year)
Net, first year			**236 h (13× the build time)**
Average per month (form field 13; frequency: monthly)			**~20 h a month**

Where 30 hours a run comes from

Manual step, per consolidation run	Hours
Export, map to budget lines and tie about 34 Yardi and QBO files to their own totals (about 30 minutes each)	17
Tie and eliminate intercompany fees between the funds, properties and management company	4
Find accruals, catch-ups, capital items in repairs and stopped books by eye	5
Rebuild the consolidated tables and the deck	4
Total	**30**

Range

Case	Assumptions	Saved per month	Payback on 18 h
Low	20 h manual vs 6 h per run; 10 h manual vs 4 h per monthly refresh; 1 h a month upkeep	~10 h	under 2 months
Base	As above	**~20 h**	**the first run (saves 26 h)**
High	Base plus about 4 scenario requests a month (2 h each by spreadsheet vs 15 minutes on sliders) and one mid-year reforecast run	~29 h	the first run

Why it holds up. The manual baseline is observable: the real run read 34 exports and checked 3,042 source totals, and the budget calendar needs four consolidation runs. The automated side keeps the human work in: exports, the CFO's rulings on flagged items, and sign-off. Upkeep is netted out. Even the Low case pays back within two months. And the claim will be measured: the Nov 3 run is timed and logged against the 30-hour baseline, and the figure is reviewed quarterly with the AI governance review.

Not counted: errors caught before they reach the budget (all 25 planted problems in the test), live scenario decisions at the offsite instead of a week-later rework, and one source of truth across two ledgers. To put a dollar value on it, multiply the hours by the loaded hourly cost of the staff who do the work today.

10. The agent swarm: Analyst, Skeptic, CFO

This debate was simulated in a Claude session on the app's Base case and the forecast outputs, following the brief's three-agent workflow. Each agent saw the same files.

Agent A, the Analyst: "Here's the number"

FY2027 Base: consolidated revenue $14.06M, NOI $5.36M, net income $1.00M. Bridge from TTM NOI of $3.88M:

- momentum baseline +$0.10M
- rent growth +$0.31M
- opex inflation −$0.25M
- deployment of the new raise +$1.31M

The property baseline is solid: 6 of 8 properties are built entirely from their own months, and the backtest error is 5.9% (seasonal) and 1.5% (steady).

Agent B, the Skeptic: "Here's what's wrong with it"

1. **A quarter of Base NOI ($1.31M) comes from money not yet raised or invested.** It assumes $25M raised, 60% deployed at a 7% cap and 60% LTV, with half a year of ownership. Under our own rule, items under about 50% probability belong on the R&O list, not in Base.
2. **The backtest is flattered.** The demo parks share one seasonal shape by construction (5.9%); real books scored 36% in Week 5. The ±10% band on own months is a judgment, not a measurement.
3. **The baseline is PROVISIONAL.** Six checks are still open, including 13 missing uploads and an untied intercompany difference. The forecast used proposed adjustments as if they were approved.
4. **Two properties rest on assumptions.** Elm Street's lease-up run-rate (+21% on its TTM revenue) assumes August occupancy holds, and its September books stopped. Bluewater has 7 of 12 months borrowed, and its contract billings before we owned it are not assumed at all.
5. **One inflation rate for every cost** understates property tax and insurance resets.

Agent C, the CFO: "Here's the decision"

- **Adopt the momentum baseline as every property's starting point.** It is backtested, explainable, and inherits the cleaning. Budget = baseline + drivers + owned initiatives.
- **Move deployment out of Base and into R&O** until a deal is under LOI. Base consolidated NOI becomes $4.04M and net income $0.42M. Raise-pace fees stay in Base only for signed or soft-circled commitments; IR confirms by the Nov 3 checkpoint.
- **Nothing leaves Finance while PROVISIONAL.** Re-run on real data with approved adjustments only, once the missing uploads arrive.
- **Asset Management's first two questions:** Bluewater's contract calendar before acquisition, and Elm Street's lease-up path. Add a separate inflation line for property tax and insurance.
- **Track it during the year.** Monthly occupancy and transient revenue against the baseline band; a deal-pipeline tracker for deployment; the forecast backtest re-run at each close.

11. How it was built: timeline, SNAKE prompt and iteration log

When	What happened
Weeks 3–5	Reusable AI workflows on our own work: a Claude skill for the 13-week cash forecast (Week 3), fee-recovery automation (Week 4, whose "fees booked Day 1 of each quarter" rule became the intercompany tie), and a seasonal RV-park baseline that borrows peers' seasonality (Week 5).
Oct 20–21	Project set-up for the FY2027 budget: instructions, roadmap, a best-practice playbook (Wattig), BIWS Excel standards and the capstone sprint plan.
Oct 22	Kickoff with Asset Management and the property budget template. Capstone plan reset to run on real books, anonymized only for submission.
Oct 23	A read-only diagnosis of the real files (16 findings), then the 14-rule cleaning rulebook, approved before any code. BudgetPipeline v0.1 (3 loaders, 12 detectors, adjustments register, checks, BIWS

When	What happened
	workbook) and v0.2 (completeness gate: a P&L and a balance sheet for every entity-month).
Oct 24	v0.2.1 fixed a live Colab error. v0.2.3 ran on 34 real Yardi and QBO exports, with 3,042 of 3,042 source totals tying and one book of record per entity-month. v0.2.4 added the fictional demo company: 10 files, 25 planted problems, 25 caught. The Streamlit scenario app followed: six drivers, Low/Base/High, tested.
Oct 25	v0.2.5 added the forecaster (the Week 5 method on every property). Also: the submission guide, and the app deployed from GitHub to Streamlit Cloud.

	SNAKE prompt for the Week 6 forecast extension
S	Capstone due tonight. The pipeline cleans and consolidates 13 demo entities (Oct 2025–Sep 2026); my Week 5 notebook forecast only four RV parks with peer-borrowed seasonality.
N	A baseline forecast for every property, inside the pipeline (reusable, not a one-off notebook), feeding the Streamlit app; anonymized data only.
A	Port the Week 5 method into a class: account classification, asset families, the same 7 approaches and written selection rule, Low/Base/High by property, line and month; an Excel export; app data; a notebook step with a CFO memo generated from the numbers.
K	Use the normalized baseline; never treat stopped books as zero; peers only within a family; below-NOI lines are not forecast; intercompany fees must still tie; BIWS formatting.
E	Notebook runs with 0 errors; workbook checks OK; the app's bridge still adds up; tests pass; one number checked by hand.

What I saw	Diagnosis	Fix and effect
The demo's prior-year columns would give 24 months of history	They are synthetic, derived from the actuals ×0.93: using them as history would leak	Not used; the forecast runs on 12 real months, like Week 5
Elm Street's September revenue = $0	The pipeline flags it as stopped books (D05), not a zero month	Treated as a gap and filled from steady peers (8% of its revenue)
A Week 5-style "same month last year" for apartments	Misses rent steps and lease-up	Steady family: the backtest picks the run-rate (1.5% vs 2.0% for a 3-month average)
Workbook NOI check off by $0.17	Values rounded to cents cell by cell	Write unrounded values; the check shows OK
App net income fell $0.25M on the new baseline	The old peer gross-up scaled interest with summer revenue	Kept: the line-by-line forecast is right; explained in §5
A range tile rendered as a formula	Streamlit reads $…$ as LaTeX	Dollar signs escaped

12. Loom run of show (about 6:00)

Before you hit record

- **Wake the app** 5 minutes before you record. Community Cloud apps sleep when idle; if you see "This app has gone to sleep", click the button to wake it. Set it to Base, the Bridge tab and NOI.
- **Open these browser tabs in order:**
  - 1) the guide PDF, page 1
  - 2) the GitHub repository
  - 3) the executed notebook (click Consolidation_Pipeline_Demo_executed.ipynb in the repository; GitHub shows it with outputs)
  - 4) the live app

  - Pre-scroll the notebook tab to step 2b.
- **Keep the company name off screen.** Don't show File Explorer or any file path or title bar that shows the company name. If you show a workbook, open a copy from Downloads or Google Drive. Use a browser window without work bookmarks (Ctrl+Shift+B hides the bookmarks bar). Close email and Teams.
- **Set up Loom.** Screen + Camera, full screen, microphone checked. Aim for about 6 minutes. Say the level in your first sentence.
- **Have the cue sheet in front of you** (Loom_Cue_Sheet.pdf, on a second screen or printed). Paraphrase freely; the numbers are the part to get right.

The script

Time	Show and do	Say (paraphrase freely)
0:00–0:20	Guide, page 1. Camera on.	"I'm Kurt Cherrington, CFO, and I'm submitting at the Mastery level. My capstone is the engine behind our firm's first real budget: raw ledgers in; a consolidated, cleaned, forecast and stress-tested FY2027 out. Everything you'll see runs on a fictional company with synthetic data. The same code ran on our real books."
0:20–0:55	Guide §2 (the friction table).	"The problem: about 31 legal entities in two ledgers, Yardi and QuickBooks. Fees flow between the funds, the properties and the management company. The close is being rebuilt, and several properties were bought mid-year. By hand, one trustworthy consolidated view takes days, and we need four of them before our November offsite. And nothing told our asset managers what a normal year looks like."
0:55–1:35	Guide §3 (the rules table).	"Before writing any code, I diagnosed the real files read-only and found 16 issues. Then I wrote a 14-rule cleaning rulebook and approved it before a line of code existed. Dates come in eight formats. Text in a number cell is left blank and logged, never zeroed. An empty month is classed as a real zero, a closed season, stopped books or not owned. Each entity-month has one book of record. Raw data is never edited: every fix is a reason-coded adjustment, and I approve anything over five thousand dollars."
1:35–2:30	GitHub repository (one second), then the executed notebook. Scroll from the step 2b grid down to step 6, "Caught 25 of 25".	"That rulebook became BudgetPipeline, a class-based package: load, clean, map, detect, consolidate, check, export. Next month I drop in the new exports and press Run all. It won't run quietly on incomplete data: this grid shows every entity and month that was uploaded. To prove it works, the demo files carry 25 planted problems, like a missing water bill, an unbilled tenant and a roof expensed to repairs. It catches all 25. On our real exports, 3,042 of 3,042 source totals tie to the penny."
2:30–3:30	Notebook step 4c: the leaderboard chart, then the CFO memo.	"Then the forecast. In Week 5, I built a seasonal baseline for four RV parks; here it runs on every property. The hard part is the months a property hasn't operated under our ownership. I backtested seven approaches by hiding months. Prophet on a property's own data misses by 74%. Borrowing a peer's seasonal shape misses by 6%, and for apartments and industrial a simple run-rate wins at 1.5%. The most accurate approach is also the one a board can follow in a sentence. So a written rule picks the simpler model unless a fancier one clearly wins. That's my accuracy-versus-interpretability answer for a CFO."
3:30–4:40	Live app. (1) Point at the tiles and the bridge. (2) Drag Rent growth from 3% to 5% and watch the bridge. (3) Click Low, then High. (4) Baseline forecast tab: pick Bluewater Marina & RV. (5) Flash the Data quality tab.	"This is the app, live on Streamlit Cloud. It starts from that baseline, and six sliders move the budget: occupancy, rent growth, opex inflation, rates, raise pace and deployment. Each bar in the bridge is one driver's dollars, and they add up exactly; that's tested. Base NOI is 5.36 million, with Low at 3.65 and High at 7.42. Here's Bluewater, bought in May: the open circles are the only months we've owned it, and the rest is borrowed from its peers. The data-quality tab keeps it honest: while any check is open, the baseline is stamped provisional."
4:40–5:10	Guide §10 (agent swarm).	"I stress-tested the draft with three agents. The Analyst presented Base. The Skeptic pointed out that 1.3 million of Base NOI comes from capital

Time	Show and do	Say (paraphrase freely)
		we haven't raised or deployed yet. As CFO, I moved deployment to risks and opportunities until a deal is under LOI, which brings Base NOI to 4.04 million."
5:10–5:40	Guide §7 (scalability table), then §9 (ROI).	"What breaks if the data doubles? I measured it. The pipeline and app stay linear, and the backtest goes from 40 to 80 seconds. What breaks first is people: new config rows and more findings to review, so only material items come to me. Security: there's no personal data, the submission is synthetic, and real data stays in approved tools. ROI: it took 18 hours to build and saves about 20 hours a month, net of upkeep, so the first real consolidation run on November 3rd pays it back."
5:40–6:00	Guide §1, the Links row.	"The app, the code on GitHub and a Google Drive backup are all linked in the submission. Next step: the same engine consolidates our asset managers' real submissions on November 3rd. Thank you."

13. Before you submit

1. **Deploy the app (done).** Live at https://fy2027-budget-scenarios-nvx3sn4bhmgkrtyz5vgpjt.streamlit.app/ from the public repository github.com/kurtcherrington-boop/fy2027-budget-scenarios. Open the link in an incognito window and move one slider to confirm a grader can use it without your login.
2. **Run the notebook in Colab.** Upload Consolidation_Pipeline_Demo.ipynb, choose Runtime > Run all, and upload Demo_Upload_Files.zip at step 1. You should see "Caught 25 of 25" in step 6 and, in step 4c, "Selected approach: Seasonal = Single-peer ratio, Steady = Run-rate", with $11.36M revenue and $5.36M NOI in the bottom line.
3. **Record the Loom.** Follow §12 (about 6 minutes), and state the level first. Loom_Cue_Sheet.pdf is the same script as a printable cue sheet.
4. **Fill in the form numbers.** Field 12, build time: 18 hours. Field 13, ROI: 20 hours. Field 14: Monthly. Field 15, other impact: the "Not counted" paragraph in §9. Field 6: the abstract in the appendix.
5. **Submit via the course form.** The form takes links, not files. The anonymized submission files sit in the GitHub repository and in the Google Drive backup folder (set the folder to "Anyone with the link: Viewer"). Put all three links from §1 in field 8: the app, the repository and the Drive folder. Do not upload Demo_Build_Internal_NOT_FOR_SUBMISSION.zip or anything from the Consolidation_Pipeline folder (real data).

Appendix: abstract for the form (466 words, links included)

Our private-equity real estate firm is building its first real budget (FY2027) from about 31 legal entities kept in two ledgers, Yardi and QuickBooks Online. Fees flow between funds, properties and the management company, the close is being rebuilt, and several properties were bought mid-year. By hand, one trustworthy consolidated view takes about 30 hours, and we need four before year-end.

Cleaning logic before code. After a read-only diagnosis of the real files found 16 issues, I wrote and approved a 14-rule cleaning rulebook before writing any code. Dates come in eight formats. Text in a number cell is left blank and logged, never zeroed. An empty month is classed as a real zero, a closed season, stopped books or not owned. Each entity-month has one book of record. Raw data is never edited: each fix is a reason-coded adjustment, and I approve anything over $5,000.

Reusable pipeline. The rules became BudgetPipeline, a class-based Python package. It loads raw Yardi and QuickBooks exports, cleans and maps them, runs twelve anomaly detectors, consolidates with

intercompany elimination and checks itself. Next month needs only new exports and Run all. On 34 real exports, 3,042 of 3,042 source totals tie. On a fictional demo company, it catches 25 of 25 planted problems. 73 automated tests back it.

Forecast and model trade-off. BaselineForecaster extends my Week 5 project to every property: an FY2027 momentum baseline by property and month. I backtested seven approaches by hiding months. Prophet on a property's own data missed by 74%. Borrowing a peer's seasonal shape missed by 6%, and a simple run-rate missed by 1.5% for apartments and industrial. The most accurate approach was also the easiest to explain, so a written rule keeps the simpler model unless a complex one clearly wins.

Interactive app. A Streamlit app starts from that baseline. Six sliders (occupancy, rent growth, opex inflation, interest rates, raise pace and deployment) move the budget, and a bridge shows each driver's dollars. Base NOI is $5.36M, with Low at $3.65M and High at $7.42M. An Analyst-Skeptic-CFO agent debate moved $1.3M of NOI that depends on unraised capital out of Base.

Scalability, security and ROI. At double the data, the pipeline and app stay linear, and the backtest goes from 40 to 80 seconds. What breaks first is human review, so only material items are routed to me. The submission is synthetic; real data stays in approved tools, and no personal data is in scope. Built in about 18 hours, it saves about 20 hours a month net of upkeep and pays for itself on the first real consolidation run, which I will time against the 30-hour baseline.

Next, the same engine consolidates our asset managers' real submissions on November 3.

Links

Live app: https://fy2027-budget-scenarios-nvx3sn4bhmgkrtyz5vgpjt.streamlit.app/

Code and files: https://github.com/kurtcherrington-boop/fy2027-budget-scenarios

Backup (Google Drive): https://drive.google.com/drive/folders/1UlcjnGKsTGNs0rXDICw5qK7F902WrZnh?usp=drive_link